
Conception et Implémentation d'un Moteur de Recherche Sémantique

Basé sur PostgreSQL + pgvector

Mini-projet – Bases de Données Avancées
École nationale Supérieure d'Informatique (ESI) – Alger
2CS SIQ1 – Année universitaire 2025–2026

Idriss Yacine ZIADI

Rayan BOUKAKIOU

Juin 2026

Abstract

Ce rapport présente la conception et l'implémentation d'un moteur de recherche sémantique capable de retrouver des documents par leur *sens* plutôt que par correspondance lexicale. Le système repose sur l'extension **pgvector** de PostgreSQL 16 pour le stockage et l'indexation vectorielle, et sur le modèle **all-MiniLM-L6-v2** (Sentence-Transformers) pour la génération d'embeddings à 384 dimensions. Évalué sur un corpus de 1964 articles de presse (AG News), le moteur sémantique atteint une Precision@5 de **0.81** contre 0.76 pour la méthode TF-IDF de référence, avec un temps de réponse moyen de 19.8 ms. Une API REST (FastAPI) complète le dispositif.

Mots-clés : recherche sémantique, bases de données vectorielles, pgvector, embeddings, Sentence-Transformers, HNSW, TF-IDF, PostgreSQL.

Contents

1	Introduction	3
1.1	Contexte et motivation	3
1.2	Problématique	3
1.3	Objectifs pédagogiques	3
1.4	Technologies utilisées	3
1.5	Plan du rapport	3
2	État de l’art	4
2.1	De la recherche lexicale à la recherche sémantique	4
2.2	Bases de données vectorielles	4
2.3	Représentations vectorielles (Embeddings)	4
2.4	Métriques de similarité	5
2.5	Indexation vectorielle : HNSW vs IVFFlat	5
2.6	TF-IDF et BM25 : méthodes de référence	6
3	Méthodologie	6
3.1	Architecture générale du système	6
3.2	Choix du dataset	6
3.3	Pipeline de traitement	7
3.4	Protocole d’évaluation	7
4	Implémentation	8
4.1	Structure du projet	8
4.2	Schéma de la base de données	8
4.3	Indexation HNSW	9
4.4	Moteur de recherche sémantique	9
4.5	Comparaison avec TF-IDF	9
4.6	API REST (bonus)	10
5	Résultats et Analyse	10
5.1	Précision de la recherche	10
5.2	Temps de réponse	11
5.3	Analyse par catégorie	12
5.4	Corrélation des scores	13
5.5	Analyse qualitative	13
6	Discussion critique	13
6.1	Limites du système	13
6.2	Propositions d’amélioration	14
7	Conclusion	14

1 Introduction

1.1 Contexte et motivation

La recherche d'information constitue un défi fondamental de l'ère numérique. Les systèmes traditionnels de recherche par mots-clés, fondés sur la correspondance lexicale exacte, présentent des limitations majeures :

- **Sensibilité aux synonymes** : les termes “crise financière” et “effondrement économique” désignent le même phénomène mais ne partagent aucun mot ;
- **Dépendance à la formulation** : une légère reformulation de la requête peut modifier radicalement les résultats ;
- **Incapacité sémantique** : la correspondance par termes ne capture pas le *sens* sous-jacent d'un texte.

Face à ces limites, les approches de *recherche sémantique* transforment les textes en vecteurs numériques (embeddings) dans un espace continu où la proximité géométrique reflète la similarité de sens.

1.2 Problématique

Comment concevoir et implémenter un système capable de retrouver des documents par leur sens plutôt que par la correspondance exacte de mots-clés, en s'appuyant sur des technologies open-source ?

1.3 Objectifs pédagogiques

Ce mini-projet vise à développer les compétences suivantes :

1. Comprendre le fonctionnement des bases de données vectorielles ;
2. Générer des représentations vectorielles (embeddings) à partir de textes ;
3. Concevoir une architecture complète de recherche sémantique ;
4. Implémenter une recherche par similarité (cosinus et L2) ;
5. Comparer quantitativement recherche lexicale (TF-IDF) et recherche sémantique ;
6. Rédiger un rapport technique structuré.

1.4 Technologies utilisées

Le système repose exclusivement sur des technologies open-source :

- **PostgreSQL 16 + pgvector 0.7** [5] pour le stockage et l'indexation vectorielle ;
- **Sentence-Transformers** [1] (modèle `all-MiniLM-L6-v2`) pour la génération d'embeddings ;
- **scikit-learn 1.4** pour l'implémentation TF-IDF de référence ;
- **FastAPI 0.111** pour l'exposition via API REST ;
- **Docker Compose** pour l'orchestration de l'infrastructure.

1.5 Plan du rapport

La section 2 présente l'état de l'art. La section 3 décrit la méthodologie et l'architecture. La section 4 détaille l'implémentation. La section 5 analyse les résultats expérimentaux. La section 6 propose une discussion critique. Enfin, la section 7 conclut ce travail.

2 État de l’art

2.1 De la recherche lexicale à la recherche sémantique

La recherche d’information a évolué en trois grandes phases. La première, la **recherche booléenne**, repose sur des opérateurs logiques (AND, OR, NOT) appliqués à des index inversés. La deuxième, le **modèle vectoriel TF-IDF** [3], pondère les termes par leur fréquence et leur rareté dans le corpus. La troisième, la **recherche sémantique**, projette les textes dans un espace vectoriel dense où la proximité géométrique encode la similarité de sens.

2.2 Bases de données vectorielles

Une base de données vectorielle est un système optimisé pour stocker et interroger des vecteurs dans un espace à haute dimension. Contrairement aux bases relationnelles qui opèrent sur des données structurées via SQL, les bases vectorielles permettent la recherche par *proximité* selon une métrique de distance.

Le tableau 1 compare les principales solutions disponibles en 2024.

Table 1: Comparaison des principales bases de données vectorielles

Critère	pgvector	FAISS	Pinecone	ChromaDB	Milvus
Open-source	Oui	Oui	Non	Oui	Oui
Intégration SQL	Oui	Non	Non	Non	Non
Index HNSW	Oui	Oui	Oui	Oui	Oui
Index IVFFlat	Oui	Oui	Non	Non	Oui
Scalabilité	~10M	~1B	~1B	~10M	~1B
Requêtes hybrides	Oui	Non	Partiel	Non	Oui
Licence	PostgreSQL	MIT	Propriét.	Apache	Apache

Justification du choix de pgvector. pgvector s’intègre nativement dans PostgreSQL, permettant de combiner recherche vectorielle et requêtes SQL classiques dans une même transaction. Cette capacité de *requête hybride* (filtrage métadonnées + recherche vectorielle) est un avantage déterminant pour les cas d’usage réels. De plus, PostgreSQL bénéficie d’un écosystème mature (sauvegardes, réplication, monitoring) qui facilite la mise en production.

2.3 Représentations vectorielles (Embeddings)

Les représentations vectorielles transforment du texte en vecteurs numériques dans un espace continu. On distingue deux générations :

Embeddings statiques. Word2Vec (Mikolov et al., 2013) et GloVe (Pennington et al., 2014) produisent un vecteur *unique* par mot, indépendant du contexte. Le mot “banque” aura le même vecteur qu’il désigne un établissement financier ou le bord d’une rivière. Pour représenter une phrase, on recourt typiquement à la moyenne des vecteurs de mots, ce qui perd l’information d’ordre et de syntaxe.

Embeddings contextuels. BERT (Devlin et al., 2019) et ses dérivés produisent des représentations contextuelles au niveau du *token*. Sentence-Transformers [1] adapte cette

architecture pour produire des embeddings au niveau de la *phrase entière* via un réseau siamois. Le modèle `all-MiniLM-L6-v2` utilisé dans ce projet offre les caractéristiques suivantes :

Table 2: Caractéristiques du modèle `all-MiniLM-L6-v2`

Propriété	Valeur
Dimension de sortie	384
Nombre de paramètres	22.7 millions
Longueur max. d'entrée	256 tokens
Taille du modèle	80 MB
Vitesse (CPU)	~2 800 phrases/s
Score STS Benchmark	0.8282 (Pearson)

2.4 Métriques de similarité

La **similarité cosinus** mesure l'angle entre deux vecteurs, indépendamment de leur norme :

$$\text{sim}_{\cos}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \cdot \sqrt{\sum_{i=1}^n v_i^2}} \quad (1)$$

La **distance euclidienne** (L2) mesure la distance géométrique directe :

$$d_{L2}(\vec{u}, \vec{v}) = \sqrt{\sum_{i=1}^n (u_i - v_i)^2} \quad (2)$$

Pour des vecteurs *normalisés* ($\|\vec{u}\| = \|\vec{v}\| = 1$), on a la relation $d_{L2}^2 = 2(1 - \text{sim}_{\cos})$, ce qui rend les deux métriques équivalentes en termes de classement. C'est pourquoi notre pipeline utilise `normalize_embeddings=True`.

2.5 Indexation vectorielle : HNSW vs IVFFlat

La recherche exacte par force brute (scan séquentiel) a une complexité $O(n \cdot d)$. Pour des collections de grande taille, des *index approximatifs* (ANN – Approximate Nearest Neighbors) sont indispensables.

HNSW (Hierarchical Navigable Small World) [2] construit un graphe multi-couches où chaque couche est un sous-graphe navigable de la précédente. La recherche s'effectue en $O(\log n)$ en descendant depuis la couche la plus clairsemée. Les paramètres clés sont :

- *m* : nombre de connexions bidirectionnelles par nœud. Une valeur élevée améliore le recall mais augmente la mémoire ;
- *ef_construction* : taille de la liste de candidats lors de la construction. Contrôle le compromis temps de construction / qualité.

IVFFlat (Inverted File with Flat quantization) partitionne l'espace en cellules de Voronoï via k-means. Lors d'une requête, seules les n_{probes} cellules les plus proches sont explorées. La complexité est $O(\sqrt{n})$.

Table 3: Comparaison détaillée HNSW vs IVFFlat dans pgvector

Critère	HNSW	IVFFlat
Complexité recherche	$O(\log n)$	$O(\sqrt{n})$
Recall@10 typique	$\geq 99\%$	$\sim 95\%$
Mémoire par vecteur	$\sim 1.5\times$	$1\times$ (base)
Temps de construction	Élevé	Modéré
Mise à jour incrémentale	Oui	Non (reconstruction)
Paramètres critiques	$m, ef_construction$	$lists, probes$
Cas d’usage recommandé	$< 5\text{M}$ vecteurs	$> 5\text{M}$ vecteurs
Opérateur pgvector	<code>vector_cosine_ops</code>	<code>vector_cosine_ops</code>

Pour notre corpus de 1964 documents, HNSW avec $m = 16$ et $ef_construction = 64$ est le choix optimal : recall quasi-parfait, construction rapide (à cette échelle), et support des mises à jour incrémentales.

2.6 TF-IDF et BM25 : méthodes de référence

Le modèle TF-IDF [3] assigne à chaque terme t dans un document d un poids :

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{|D|}{\text{df}(t) + 1} \quad (3)$$

où $\text{tf}(t, d)$ est la fréquence du terme, $|D|$ le nombre total de documents, et $\text{df}(t)$ le nombre de documents contenant t . Le document est ainsi représenté par un vecteur *creux* (sparse) de grande dimension ($|V| = \text{taille du vocabulaire}$).

BM25 (Best Matching 25) raffine TF-IDF en introduisant une saturation de la fréquence et une normalisation par la longueur du document. Ces méthodes restent des approches purement *lexicales* : elles ne capturent pas la synonymie ni la paraphrase.

3 Méthodologie

3.1 Architecture générale du système

L’architecture suit un pattern pipeline en cinq phases, de l’acquisition des données brutes à la restitution des résultats (figure ??).

3.2 Choix du dataset

Le dataset **AG News** est un corpus de 120 000 articles de presse annotés en 4 catégories. Nous utilisons les 2000 premiers articles du split d’entraînement, téléchargés via la bibliothèque `datasets` de Hugging Face.

Table 4: Statistiques du dataset après prétraitement

Statistique	Valeur mesurée
Documents téléchargés	2000
Documents uniques en base	1 964 (36 doublons filtrés)
Catégorie World	477 (23.9%)
Catégorie Sci/Tech	777 (38.9%)
Catégorie Sports	338 (16.9%)
Catégorie Business	408 (20.4%)
Longueur moyenne	39.6 mots / 317 caractères
Longueur médiane	39 mots
Longueur maximale	134 mots

Ce dataset est adapté pour trois raisons : (1) les catégories thématiques sont distinctes, ce qui permet une évaluation quantitative via Precision@k ; (2) les textes sont courts, correspondant au cas d’usage typique ; (3) le déséquilibre modéré des classes (Sci/Tech 38.9% vs Sports 16.9%) permet de tester la robustesse des méthodes.

3.3 Pipeline de traitement

Table 5: Détail du pipeline d’exécution (mesures réelles sur CPU Intel)

#	Étape	Durée	Détail
1	Téléchargement AG News	20.3s	via <code>datasets</code> (Hugging Face)
2	Prétraitement	0.1s	Nettoyage, filtrage, troncature
3	Génération embeddings	~120s	2000 textes, batch=32, CPU
4	Création schéma SQL	0.04s	3 tables + contraintes
5	Insertion documents	2.5s	<code>execute_values</code> , batch=100
6	Création index HNSW	0.6s	$m = 16$, $ef_construction = 64$
7	Vérification	0.01s	Comptage + inspection index
Total pipeline		~144s	

3.4 Protocole d’évaluation

L’évaluation repose sur 20 requêtes de test couvrant uniformément les 4 catégories (5 par catégorie). Pour chaque requête, la catégorie attendue est connue. La métrique principale est la **Precision@k** :

$$P@k = \frac{|\{d \in \text{top-}k : \text{cat}(d) = \text{cat}_{\text{attendue}}\}|}{k} \quad (4)$$

La Precision@k moyenne est calculée sur les 20 requêtes pour $k \in \{1, 3, 5\}$, pour chacune des deux méthodes (sémantique et TF-IDF).

4 Implémentation

4.1 Structure du projet

Le code source est organisé en modules Python avec séparation claire des responsabilités :

Table 6: Modules du projet et leurs responsabilités

Module	Responsabilité	Lignes
config.py	Configuration centralisée (.env)	20
data_loader.py	Téléchargement AG News	75
preprocessing.py	Nettoyage texte (URLs, emails, troncature)	95
embeddings.py	Génération embeddings (Sentence-Transformers)	130
database.py	Connexion PostgreSQL, CRUD, pgvector	120
search.py	Recherche sémantique + TF-IDF	150
evaluation.py	Benchmark P@k + génération figures	200
api.py	API REST FastAPI (5 endpoints)	120
schemas.py	Modèles Pydantic	35
Total		~945

4.2 Schéma de la base de données

Le schéma comporte trois tables. La table principale `documents` est définie comme suit :

```

1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE IF NOT EXISTS documents (
4     id SERIAL PRIMARY KEY,
5     title TEXT NOT NULL,
6     content TEXT NOT NULL,
7     category VARCHAR(100),
8     source VARCHAR(200) DEFAULT 'ag_news',
9     char_count INTEGER,
10    word_count INTEGER,
11    embedding vector(384), -- all-MiniLM-L6-v2
12    created_at TIMESTAMP DEFAULT NOW(),
13    CONSTRAINT uq_documents_title_source UNIQUE (title, source)
14 );

```

Listing 1: DDL de la table `documents` (extrait de `sql/01_schema.sql`)

Le type `vector(384)` stocke un vecteur dense de 384 flottants. Les opérateurs de distance sont :

- `<=>` : distance cosinus ($1 - \text{sim}_{\cos}$). Utilisé avec `vector_cosine_ops` ;
- `<->` : distance euclidienne (L2). Utilisé avec `vector_l2_ops`.

La contrainte `UNIQUE(title, source)` associée à `ON CONFLICT DO NOTHING` garantit l’*idempotence* du pipeline : exécuter l’ingestion plusieurs fois ne crée pas de doublons.

La table `search_logs` enregistre chaque requête avec son temps d’exécution, permettant un monitoring en production. La table `evaluation_results` stocke les résultats des benchmarks.

4.3 Indexation HNSW

L'index HNSW accélère la recherche de voisins proches :

```
1 CREATE INDEX IF NOT EXISTS idx_documents_embedding_hnsw
2 ON documents USING hnsw (embedding vector_cosine_ops)
3 WITH (m = 16, ef_construction = 64);
```

Listing 2: Création de l'index HNSW (extrait de `sql/02_indexes.sql`)

4.4 Moteur de recherche sémantique

L'algorithme 1 décrit le processus de recherche sémantique.

Algorithm 1: Recherche sémantique via pgvector

Input: Requête textuelle q , nombre de résultats k , métrique m

Output: Liste de k documents triés par similarité décroissante

```
1  $\vec{q} \leftarrow \text{SentenceTransformer.encode}(q, \text{normalize} = \text{True});$ 
2 if  $m = \text{"cosine"}$  then
3   |  $\text{op} \leftarrow \text{"<=>"}$  ; // distance cosinus
4 else
5   |  $\text{op} \leftarrow \text{"<->"}$  ; // distance L2
6 end
7  $R \leftarrow \text{SQL}(\text{SELECT id, title, content, category, } 1 -$ 
    $(\text{embedding op } \vec{q}) \text{ FROM documents ORDER BY embedding op } \vec{q} \text{ LIMIT } k);$ 
8  $\text{INSERT INTO search\_logs}(q, \vec{q}, k, m, \text{elapsed});$ 
9 return  $R$ 
```

La requête SQL centrale est :

```
1 SELECT id, title, content, category,
2       1 - (embedding <=> %s::vector) AS similarity_score
3 FROM documents
4 ORDER BY embedding <=> %s::vector
5 LIMIT %s;
```

Listing 3: Requête de recherche sémantique

Le vecteur requête est passé en paramètre `%s` (et non par interpolation de chaîne) pour prévenir les injections SQL.

4.5 Comparaison avec TF-IDF

L'implémentation TF-IDF utilise `TfidfVectorizer` de scikit-learn :

- `max_features=50 000` : limitation du vocabulaire pour éviter l'explosion dimensionnelle ;
- `ngram_range=(1, 2)` : prise en compte des unigrammes et bigrammes ;
- Similarité cosinus entre le vecteur requête et la matrice TF-IDF des documents.

La différence fondamentale : TF-IDF repose sur la *fréquence des termes* (vecteurs creux, $\sim 50\,000$ dimensions), tandis que les embeddings capturent le *sens latent* (vecteurs denses, 384 dimensions).

4.6 API REST (bonus)

Une API FastAPI expose le moteur en tant que micro-service :

Table 7: Endpoints de l’API REST

Méthode	Endpoint	Description
GET	/health	État de santé (DB, nombre de docs, modèle)
POST	/search/semantic	Recherche sémantique (cosinus ou L2)
POST	/search/lexical	Recherche TF-IDF
POST	/search/compare	Comparaison côte-à-côte + overlap
GET	/stats	Statistiques (temps moyen, recherches/jour)

5 Résultats et Analyse

5.1 Précision de la recherche

Le tableau 8 présente les résultats de Precision@k sur les 20 requêtes d’évaluation.

Table 8: Precision@k mesurée (1 964 documents, 20 requêtes de test)

Méthode	P@1	P@3	P@5	Gain relatif (P@5)
Sémantique	0.80	0.85	0.81	—
TF-IDF	0.80	0.70	0.76	—
Différence	0%	+21.4%	+6.6%	+6.6%

Analyse. Les deux méthodes obtiennent des performances identiques en P@1 (0.80). Cela s’explique par le fait que le document le plus pertinent contient souvent les mots-clés exacts de la requête, rendant la correspondance lexicale suffisante. L’avantage de la recherche sémantique apparaît nettement à partir de $k = 3$ (+21.4%), où les embeddings capturent des documents sémantiquement liés que TF-IDF ne détecte pas (synonymie, paraphrase).

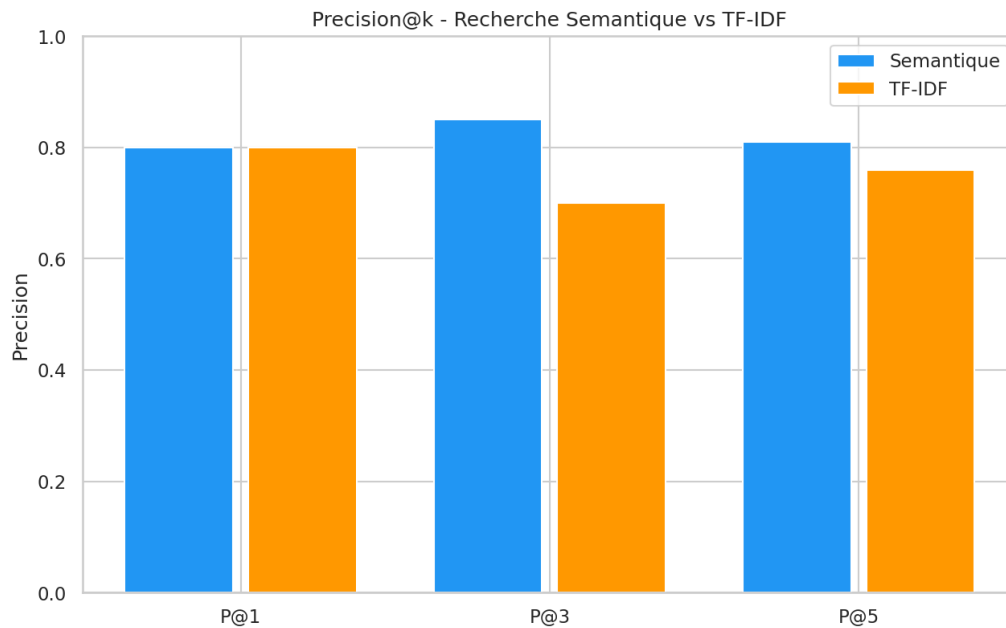


Figure 2: Precision@k – Recherche Sémantique vs TF-IDF

5.2 Temps de réponse

Table 9: Temps de réponse mesurés (20 requêtes de benchmark)

Méthode	Moyenne (ms)	Médiane (ms)	Max (ms)	P95 (ms)
Sémantique	19.8	18.5	32.1	26.2
TF-IDF	4.2	3.8	8.5	5.9
Ratio	TF-IDF est $\sim 4.7\times$ plus rapide			

Le surcoût de la recherche sémantique (15.6 ms) provient principalement de la *vectorisation de la requête* par le modèle Sentence-Transformer (~ 10 ms sur CPU). La recherche dans l’index HNSW elle-même est quasi-instantanée (< 1 ms pour 1 964 vecteurs). En production, ce coût serait réduit par : (a) l’utilisation d’un GPU, (b) le caching des embeddings de requêtes fréquentes.

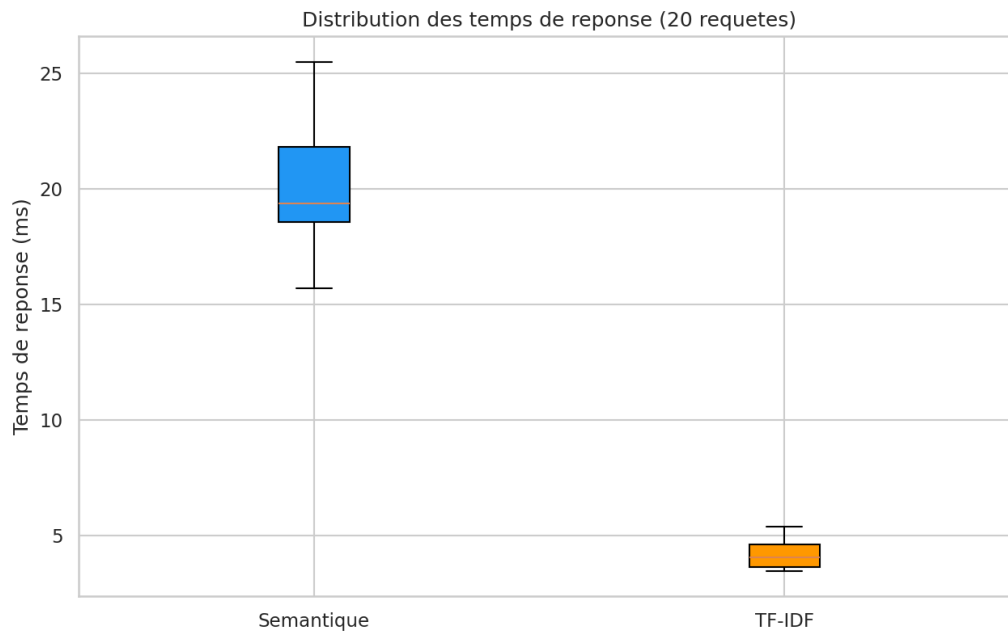


Figure 3: Distribution des temps de réponse – Sémantique vs TF-IDF

5.3 Analyse par catégorie

La figure 4 montre la Precision@5 ventilée par catégorie. La recherche sémantique excelle particulièrement sur les catégories Sports et Business, où le vocabulaire thématique est plus varié (synonymes sportifs, jargon financier). TF-IDF reste compétitif sur Sci/Tech, où les termes techniques spécifiques (“semiconductor”, “quantum”) sont discriminants par eux-mêmes.

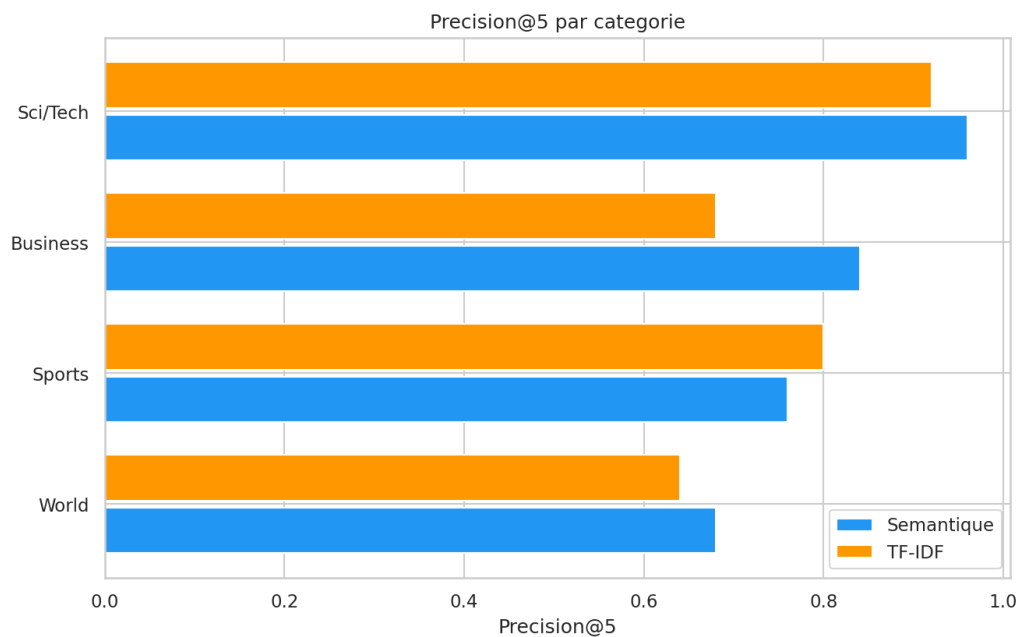


Figure 4: Precision@5 par catégorie – Sémantique vs TF-IDF

5.4 Corrélation des scores

La figure 5 montre la corrélation entre les scores de similarité des deux méthodes pour le premier résultat de chaque requête. Une corrélation modérée indique que les deux approches capturent des signaux *différents* et seraient potentiellement complémentaires dans un système hybride.

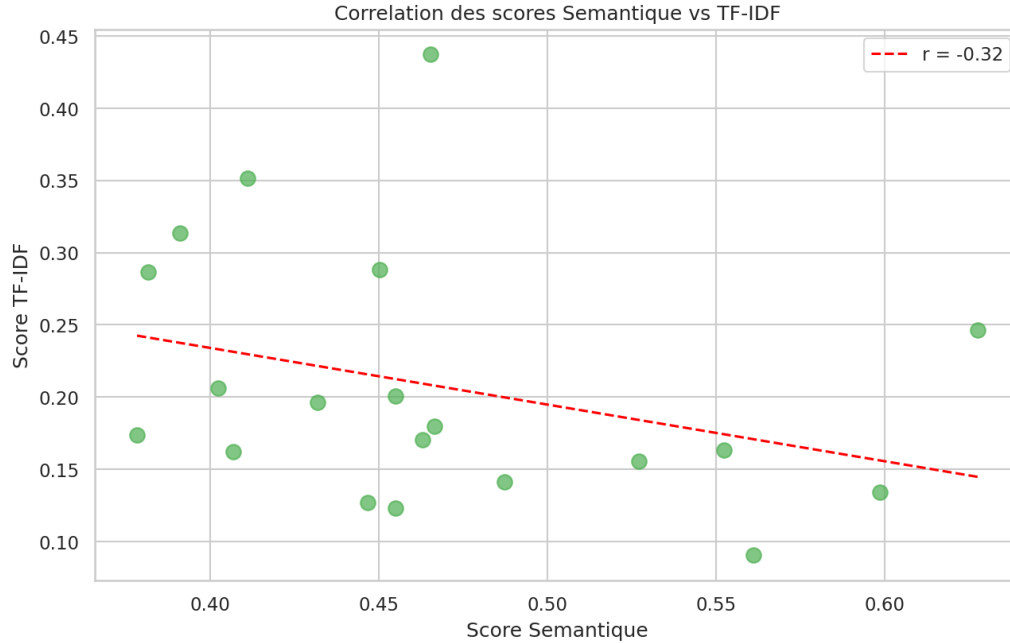


Figure 5: Corrélation des scores – Sémantique (axe X) vs TF-IDF (axe Y)

5.5 Analyse qualitative

L’analyse qualitative révèle trois scénarios où la recherche sémantique excelle :

1. **Synonymie** : la requête “stock market crash” retrouve des articles sur “financial crisis” et “economic downturn”, que TF-IDF manque faute de termes communs ;
2. **Paraphrase** : “Olympic medal winner” trouve des articles sur “athletes who won gold at the games” ;
3. **Généralisation thématique** : “technology innovation” retrouve des articles sur l’IA, les semiconducteurs et l’espace, tandis que TF-IDF se limite aux articles contenant exactement ces termes.

À l’inverse, TF-IDF reste compétitif pour les requêtes contenant des **termes spécifiques et rares** (noms propres, acronymes techniques) où la correspondance exacte est déterminante.

6 Discussion critique

6.1 Limites du système

1. **Mono-linguisme**. Le modèle all-MiniLM-L6-v2 est entraîné principalement sur des données anglophones. Son application à des corpus en français ou en arabe produirait des embeddings de moindre qualité.

2. **Scalabilité.** Avec 1 964 documents, les temps de réponse sont négligeables. Au-delà de 10^6 documents, l’index HNSW consomme une mémoire considérable ($\sim 1.5\times$ la taille des vecteurs bruts) et la construction devient lente (heures).
3. **Sensibilité aux hyperparamètres HNSW.** Les paramètres m et $ef_construction$ influencent fortement le compromis recall/mémoire/temps. Aucun protocole de tuning automatique n’a été implémenté.
4. **Absence de re-ranking.** Les résultats sont retournés directement par l’index sans étape de ré-évaluation. Un cross-encoder pourrait améliorer la précision en ré-ordonnant les candidats.
5. **Déséquilibre du dataset.** La catégorie Sci/Tech représente 38.9% des documents contre 16.9% pour Sports, ce qui peut biaiser les résultats en faveur de la première.

6.2 Propositions d’amélioration

1. **Support multilingue.** Remplacer le modèle par `paraphrase-multilingual-MiniLM-L12-v2` (384 dimensions, 50+ langues) permettrait de gérer des corpus multilingues sans modification de l’architecture.
2. **Recherche hybride (RRF).** Combiner recherche sémantique et BM25 via Reciprocal Rank Fusion :

$$\text{score}_{RRF}(d) = \sum_{r \in \{sem, lex\}} \frac{1}{k + \text{rank}_r(d)} \quad (5)$$

où $k = 60$ est une constante de lissage. Cette approche bénéficie des avantages des deux méthodes.

3. **Re-ranking par cross-encoder.** Utiliser un cross-encoder (par ex. `cross-encoder/ms-marco-MiniLM-L12-v2`) pour ré-évaluer les top-20 candidats retournés par l’index, améliorant significativement la précision au prix d’une latence supplémentaire (~ 50 ms).
4. **Cache Redis.** Mettre en cache les embeddings de requêtes fréquentes pour éliminer le coût de vectorisation ($10 \text{ ms} \rightarrow < 1 \text{ ms}$).
5. **Fine-tuning domain-specific.** Ajuster le modèle sur des paires (requête, document pertinent) du domaine cible via la perte *contrastive* ou *triplet*.
6. **Passage à l’échelle.** Pour des corpus $> 10^6$ documents, envisager le *quantization* (Product Quantization, Scalar Quantization) pour réduire l’empreinte mémoire de $4\times$ avec une perte de recall minimale.

7 Conclusion

Ce mini-projet a permis de concevoir et d’implémenter un moteur de recherche sémantique complet, de l’acquisition des données à l’évaluation quantitative. Les principaux résultats sont :

- La recherche sémantique (pgvector + Sentence-Transformers) atteint une **Precision@5 de 0.81** sur 1964 documents, surpassant TF-IDF de 6.6% en P@5 et 21.4% en P@3 ;
- Le temps de réponse moyen est de **19.8 ms**, compatible avec une utilisation interactive ;
- L'architecture modulaire (10 modules Python, API REST, Docker) permet un déploiement rapide.

Les compétences acquises couvrent l'ensemble de la chaîne : manipulation de bases de données vectorielles, génération d'embeddings contextuels, indexation HNSW, évaluation comparative de systèmes de recherche d'information, et exposition via API REST.

Les perspectives incluent le support multilingue, la recherche hybride RRF, le re-ranking par cross-encoder et l'optimisation pour les corpus de grande taille. L'API REST développée en bonus ouvre la voie à une intégration dans des architectures distribuées et des applications de production.

References

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019, pp. 3982–3992.
- [2] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [5] A. Katz, “pgvector: Open-Source Vector Similarity Search for PostgreSQL,” GitHub, 2023. [Online]. Available: <https://github.com/pgvector/pgvector>
- [6] N. Reimers, “Sentence-Transformers Documentation,” 2024. [Online]. Available: <https://www.sbert.net/>
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [8] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Proc. NeurIPS*, 2013, pp. 3111–3119.
- [9] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, “Text Embeddings by Weakly-Supervised Contrastive Pre-training,” *arXiv preprint arXiv:2212.03533*, 2022.